



Balıkesir TeknoKent Bootcamp

SQL'e Giriş



CONTENTS

- KEYWORDS
- DATABASE OBJECTS
- DATA TYPES
- OPERATORS
- FUNCTIONS
- SQL COMMAND TYPES

KEYWORDS

- ❖ DB
- ❖ DBMS
- ❖ RDBMS
- ❖ STRUCTURED DATA
- ❖ UNSTRUCTURED DATA
- ❖ OLTP
- ❖ OLAP / DWH
- ❖ TRANSACTION

DATABASE OBJECTS

TABLE

Verileri saklayan objedir. Dikeye KOLON (ALAN) yataya ROW (kayıt) denir.

VIEW

Çok kullanılan sorguları saklayan objedir. Sadece sorguyu saklar veriyi saklamaz.

PROCEDURE

Belirli bir Kural ve sıra dahiinde çalıştırılan komutları saklayan objedir.

FUNCTION

Prosedür ile aynıdır tek farkı işlem bitiminde bir değer döndürür.

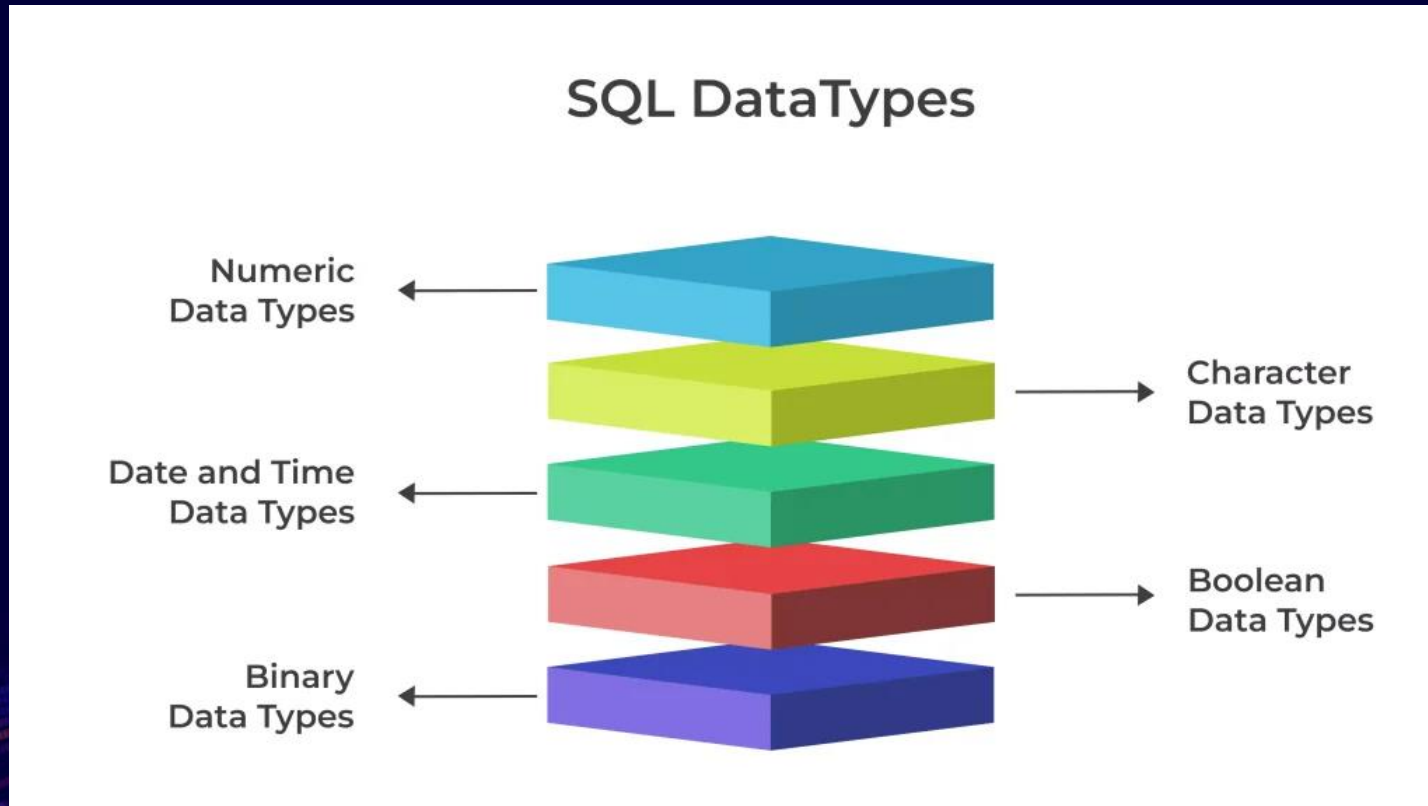
INDEX

Bir veya birden fazla kolon üzerinde verinin sıralanmasını sağlar. Sıralı alanlar üzerinden yapılan filtrelemeler daha hızlı yapılır.

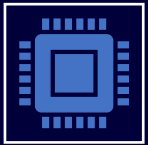
SEQUENCE

Belirli bir kural dahilinde sıra numarası üreten objedir.

DATA TYPES



DATA TYPES



Numeric Data Types:

Bu türler, tamsayılar veya ondalık sayılar gibi sayısal değerleri depolamak için kullanılır.

INTEGER
BIGINT
FLOAT
DECIMAL



Character Data Types:

Bu veri türleri, harfler, semboller ve metin gibi karakter tabanlı verileri depolamak için kullanılır.

CHAR
VARCHAR
TEXT
CLOB



Date and Time Data Types:

Bu türler tarih ve saat bilgilerini depolamak için kullanılır.

DATE
TIME
DATETIME
TIMESTAMP



Boolean Data Type:

Bu tür, doğru veya yanlış temsil eden Boolean değerlerini depolamak için kullanılır.

BOOLEAN



Binary Data Types:

Bu türler, görüntüler, ses dosyaları veya diğer binary olarak kodlanmış bilgiler gibi binary verileri depolamak için kullanılır.

BLOB
BINARY

OPERATORS

Operator	Description	Example	Result
+	addition	2 + 3	5
-	subtraction	2 - 3	-1
*	multiplication	2 * 3	6
/	division (integer division truncates results)	4 / 2	2
%	modulo (remainder)	5 % 4	1
^	exponentiation	2.0 ^ 3.0	8
/	square root	/ 25.0	5
/	cube root	/ 27.0	3
!	factorial	5 !	120
!!	factorial (prefix operator)	!! 5	120
@	absolute value	@ -5.0	5
	concatenation	'Post' 'greSQL'	PostgreSQL
^@	start with	'alphabet' ^@ 'alph'	t (true)
=	compare	'a' = 'b'	f (false)
>	compare	5 > 3	t (true)
<	compare	6 < 2	f (false)
<> veya !=	compare	7 <> 8	t (true)
IS NULL	compare	2 IS NULL	f (false)
IS NOT NULL	compare	5 IS NOT NULL	t (true)
IN	compare	5 IN (5,6,7,8)	t (true)
NOT IN	compare	6 NOT IN (1,2,3)	t (true)
LIKE – case sensitive ILIKE – case insensitive	compare	'Balıkesir' LIKE '%kes%' 'Balıkesir' LIKE '%kes_r' 'Ataşehir_İstanbul' LIKE '%_%'escape '\'	t (true)
NOT LIKE	compare	'Balıkesir' LIKE '%kes%'	f (false)
NOT	Negate (olumsuzlamak – değilini almak)	NOT 5=5	f (false)

FUNCTIONS- SCALAR

Function	Description	Example	Result
abs(x)	absolute value	abs(-17.4)	17.4
ceil(dp or numeric)	smallest integer not less than argument	ceil(-42.8)	-42
floor(dp or numeric)	largest integer not greater than argument	floor(-42.8)	-43
mod(y, x)	remainder of y/x	mod(9,4)	1
pow(a numeric, b numeric)	a raised to the power of b	pow(9.0, 3.0)	729
round(dp or numeric)	round to nearest integer	round(42.4)	42
round(v numeric, s integer)	round to s decimal places	round(42.4382, 2)	42.44
trunc(dp or numeric)	truncate toward zero	trunc(42.8)	42
trunc(v numeric, s integer)	truncate to s decimal places	trunc(42.4382, 2)	42.43
substr(string, from [, count])	Extract substring (same as substring(string from from for count))	substr('alphabet', 3, 2)	ph
strpos(string, substring)	Location of specified substring (same as position(substring in string), but note the reversed argument order)	strpos('high', 'ig')	2
length(string)	Number of characters in string	length('jose')	4

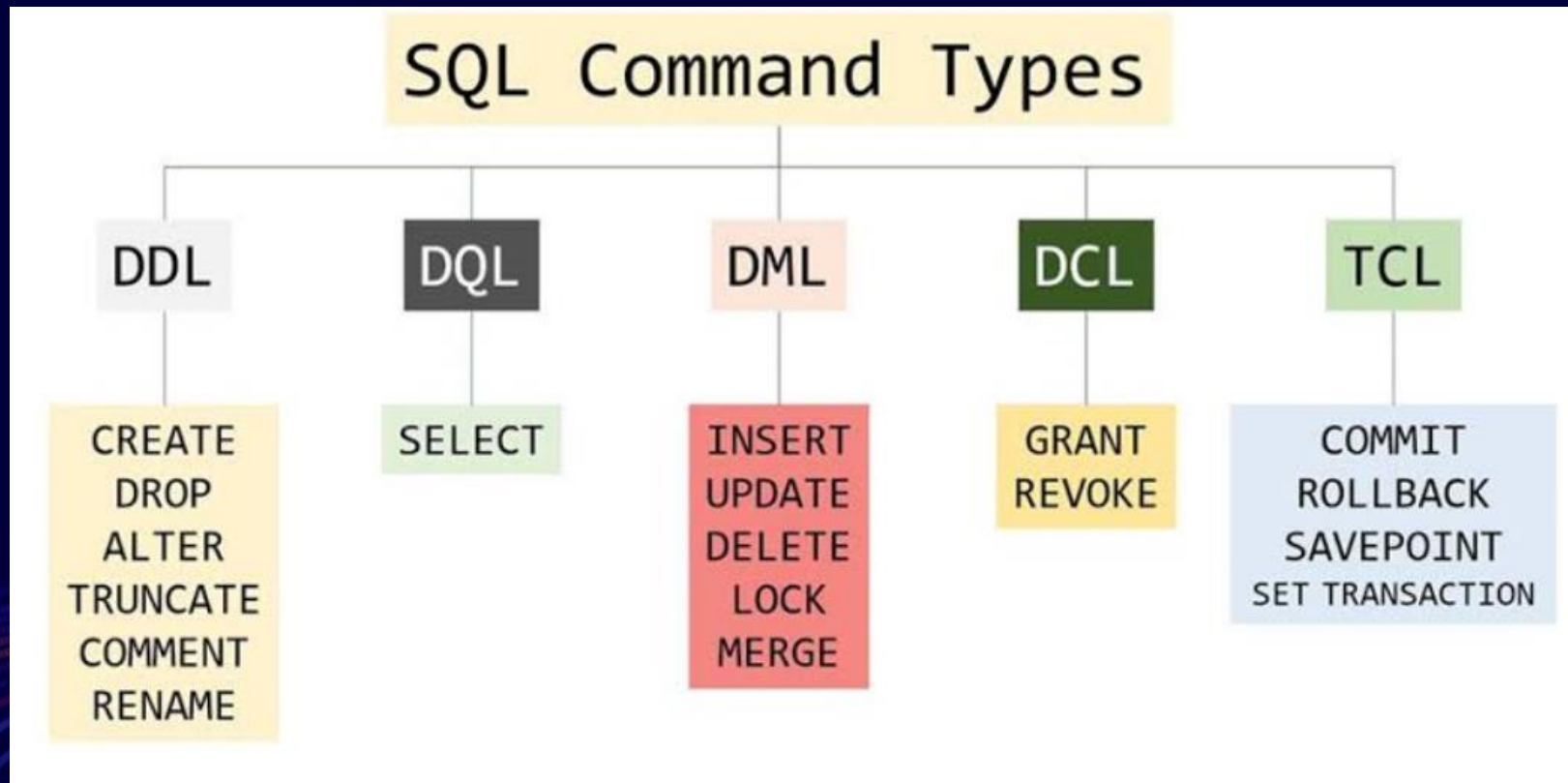
FUNCTIONS- AGGREGATE

Function	Argument Type(s)	Return Type	Description
array_agg(expression)	any	array of the argument type	input values, including nulls, concatenated into an array
avg(expression)	smallint, int, bigint, real, double precision, numeric, or interval	numeric for any integer-type argument, double precision for a floating-point argument, otherwise the same as the argument data type	the average (arithmetic mean) of all input values
count(*)		bigint	number of input rows
count(expression)	any	bigint	number of input rows for which the value of expression is not null
max(expression)	any array, numeric, string, or date/time type	same as argument type	maximum value of expression across all input values
min(expression)	any array, numeric, string, or date/time type	same as argument type	minimum value of expression across all input values
string_agg(expression, delimiter)	(text, text) or (bytea, bytea)	same as argument types	input values concatenated into a string, separated by delimiter
sum(expression)	smallint, int, bigint, real, double precision, numeric, interval, or money	bigint for smallint or int arguments, numeric for bigint arguments, otherwise the same as the argument data type	sum of expression across all input values
xmlagg(expression)	xml	xml	concatenation of XML values

FUNCTIONS- SET RETURNING

Function	Argument Type	Return Type	Description
generate_series(start, stop)	int or bigint	setof int or setof bigint (same as argument type)	Generate a series of values, from start to stop with a step size of one
generate_series(start, stop, step)	int or bigint	setof int or setof bigint (same as argument type)	Generate a series of values, from start to stop with a step size of step
generate_series(start, stop, step interval)	timestamp or timestamp with time zone	setof timestamp or setof timestamp with time zone (same as argument type)	Generate a series of values, from start to stop with a step size of step

SQL COMMAND TYPES



DDL – CREATE TABLE

Database üzerinde tablo oluşturmak için kullanılır.

```
CREATE TABLE table_name  
(  
column1 datatype,  
column2 datatype,  
column3 datatype,  
....  
);
```

```
CREATE TABLE CustomersBackup  
AS  
SELECT *  
FROM Customers  
;
```

DDL – CREATE TABLE

```
CREATE TABLE films (  
  code char(5) CONSTRAINT firstkey PRIMARY KEY,  
  title varchar(40) NOT NULL,  
  did integer NOT NULL,  
  date_prod date,  
  kind varchar(10),  
  len interval hour to minute  
);
```

```
CREATE TABLE distributors (  
  did integer PRIMARY KEY,  
  name varchar(40) NOT NULL CHECK (name <> '')  
);
```

DDL – ALTER TABLE

Tablonun tanımı üzerinde değişiklik yapmak için kullanılır.

- ☐ Yeni kolon eklemek

```
ALTER TABLE distributors ADD COLUMN address varchar(30);
```

- ☐ Var olan kolonu silmek

```
ALTER TABLE distributors DROP COLUMN address RESTRICT;
```

- ☐ Var olan kolonun adını değiştirmek

```
ALTER TABLE distributors RENAME COLUMN address TO city;
```

- ☐ Var olan kolonun veri tipini değiştirmek veya genişletmek

```
ALTER TABLE distributors  
    ALTER COLUMN address TYPE varchar(80),  
    ALTER COLUMN name TYPE varchar(100);
```

- ☐ Var olan tablonun adını değiştirmek

```
ALTER TABLE distributors RENAME TO suppliers;
```

- ☐ Var olan tabloya constraint eklemek

```
ALTER TABLE distributors ALTER COLUMN street SET NOT NULL;  
ALTER TABLE distributors ADD CONSTRAINT zipchk CHECK (char_length(zipcode) = 5);
```


DDL – DROP TABLE

Var olan tabloyu silmek için kullanılır.

- ❑ Sadece 1 tablo silmek için

```
DROP TABLE films;
```

- ❑ Aynı anda 2 tablo silmek için

```
DROP TABLE films, distributors;
```

- ❑ Tablo üzerinde tanımlı ilişkiler (relation, view) ile birlikte silmek

```
DROP TABLE films cascade;
```

DML – INSERT INTO

Var olan tabloya YENİ veri girmek için kullanılır

```
INSERT INTO films VALUES
('UA502', 'Bananas', 105, '1971-07-13', 'Comedy', '82 minutes');

INSERT INTO films (code, title, did, date_prod, kind)
VALUES ('T_601', 'Yojimbo', 106, '1961-06-16', 'Drama');

INSERT INTO films (code, title, did, date_prod, kind) VALUES
('B6717', 'Tampopo', 110, '1985-02-10', 'Comedy'),
('HG120', 'The Dinner Game', 140, DEFAULT, 'Comedy');

INSERT INTO films SELECT * FROM tmp_films WHERE date_prod < '2004-05-07';

INSERT INTO distributors (did, dname)
VALUES (5, 'Gizmo Transglobal'), (6, 'Associated Computing, Inc')
ON CONFLICT (did) DO UPDATE SET dname = EXCLUDED.dname;

INSERT INTO distributors (did, dname) VALUES (7, 'Redline GmbH')
ON CONFLICT (did) DO NOTHING;
```

DML – UPDATE

Bir tablodaki var olan kayıtları güncellemek için kullanılır

```
UPDATE films
  SET kind = 'Dramatic'
WHERE kind = 'Drama';
```

```
UPDATE weather
  SET temp_lo = temp_lo+1, temp_hi = temp_lo+15, prcp = DEFAULT
WHERE city = 'San Francisco' AND date = '2003-07-03';
```

```
UPDATE weather
  SET (temp_lo, temp_hi, prcp) = (temp_lo+1, temp_lo+15, DEFAULT)
WHERE city = 'San Francisco' AND date = '2003-07-03';
```

```
UPDATE employees
  SET sales_count = sales_count + 1
FROM accounts
WHERE accounts.name = 'Acme Corporation'
  AND employees.id = accounts.sales_person;
```


DML – MERGE

Bir tablonun başka bir veri seti ile belirli bir kolon üzerinden karşılaştırılması sonrasında INSERT/UPDATE/DELETE aksiyonları aldirmek için kullanılır.

```
MERGE INTO customer_account ca
  USING recent_transactions t
    ON t.customer_id = ca.customer_id
WHEN MATCHED THEN
  UPDATE SET balance = balance + transaction_value
WHEN NOT MATCHED THEN
  INSERT (customer_id, balance)
  VALUES (t.customer_id, t.transaction_value);

MERGE INTO customer_account ca
  USING (SELECT customer_id, transaction_value FROM recent_transactions) AS t
    ON t.customer_id = ca.customer_id
WHEN MATCHED THEN
  UPDATE SET balance = balance + transaction_value
WHEN NOT MATCHED THEN
  INSERT (customer_id, balance)
  VALUES (t.customer_id, t.transaction_value);
```

DML – DELETE

Bir tablodaki var olan kayıtları güncellemek için kullanılır

```
DELETE FROM films WHERE kind <> 'Musical';

DELETE FROM films; (DDL eşleniği TRUNCATE TABLE films; )

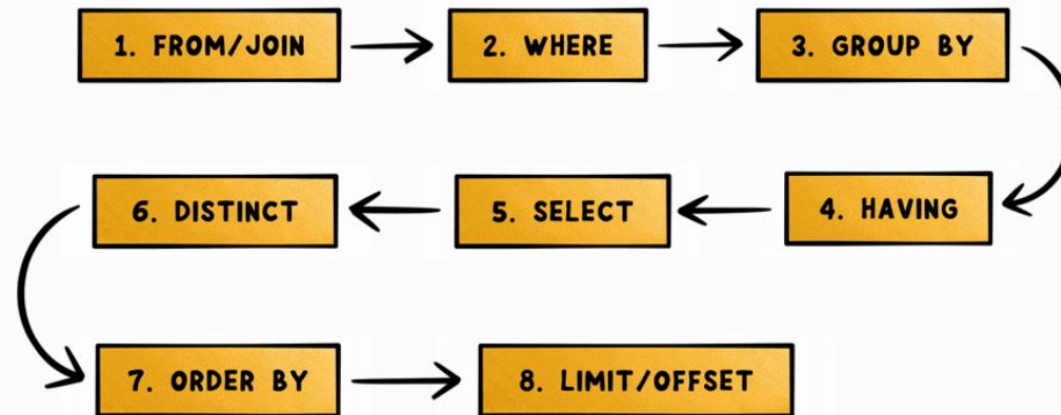
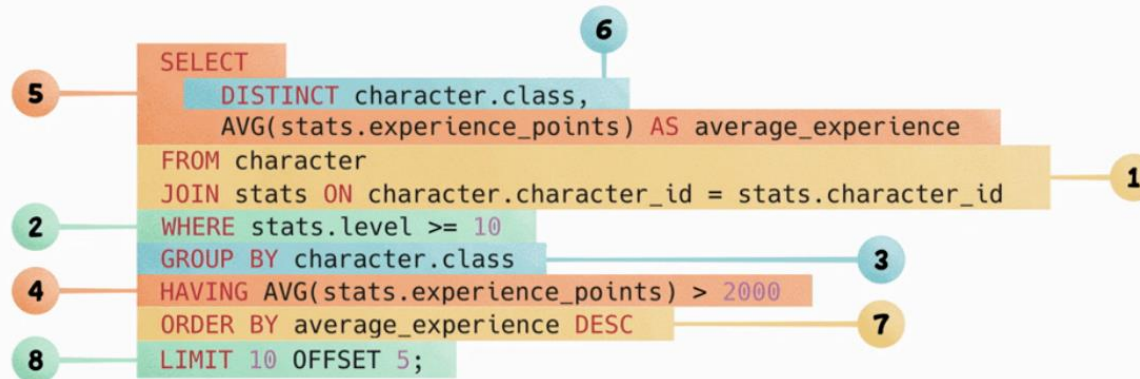
DELETE FROM films USING producers
  WHERE films.producer_id = producers.id AND producers.name = 'foo';

DELETE FROM films
  WHERE producer_id IN (SELECT id FROM producers WHERE name = 'foo');

WITH delete_batch
  AS (
    SELECT l.ctid FROM user_logs AS l
      WHERE l.status = 'archived'
      ORDER BY l.creation_date
      FOR UPDATE
      LIMIT 10000
  )
DELETE FROM user_logs AS dl
  USING delete_batch AS del
  WHERE dl.ctid = del.ctid;
```

DQL- SELECT

SQL Execution Order



WHERE

Kayıtları filtrelemek için kullanılır

```
select *  
  from humanresources.employee  
 where jobtitle='Marketing Specialist');
```

```
select *  
  from humanresources.employee  
 where jobtitle in ('Marketing Specialist','Design Engineer');
```

```
select *  
  from humanresources.employee  
 where businessentityid in (select businessentityid  
                           from humanresources.employeedepartmenthistory  
                           where enddate is null  
                             and departmentid=1);
```

```
select *  
from production.product a  
where exists (select 1  
             from product_list b  
             where a.productid=b.productid);
```

GROUP BY

Kayıtları gruplamak ve grup hesaplamaları yapmak için kullanılır

```
select jobtitle, count(*) nof_employees  
  from humanresources.employee  
 group by jobtitle;
```

HAVING

Gruplanmış kayıtları filtrelemek için kullanılır

```
select jobtitle, count(*) nof_employees
from humanresources.employee
group by jobtitle
having count(*)>10;
```

DISTINCT

- Sorgu sonucunda dönen veri kümesindeki tekrarlı kayıtları tekil hale çevirmek için kullanılır.
- Gruplama fonksiyonları fonksiyona girmeden önce distinctlemek için kullanılır.

```
select distinct jobtitle from humanresources.employee;
```

```
select count(jobtitle) total_count,  
       count(distinct jobtitle) distinct_count  
from humanresources.employee;
```


ORDER BY

Sorgu sonucunda dönen veri kümesinin istenilen kolona göre sıralamak için kullanılır.

```
select * from humanresources.employee order by hiredate;
```

LIMIT - OFFSET

Sorgu sonucunda dönen veri kümesinin belirli sayıda verisini almak için kullanılır

- Sıralama işleminden sonra ilk 10 kaydı göstermek

```
select * from humanresources.employee order by hiredate limit 10;
```

- Sıralama işleminden sonra ilk 10 kaydı atlayıp (offset) sonraki 10 kaydı göstermek

```
select * from humanresources.employee order by hiredate limit 10 offset 10;
```

UNION – UNION ALL

2 sorgu sonucunda dönen veri setini alt alta birleştirmek için kullanılır.

Önemli noktalar:

- Alt alta gelen kolonlar aynı veri tipinde olmalı
- * (star) ile yapılan union işlemlerinde kolon kaymaları olabilir. O nedenle kolon isimlerini yazmanız önerilir.
- UNION işlemi tekil değer döndürür UNION ALL ise veri içeriğine göre tekrarlı veri döndürebilir

```
SELECT City FROM Customers
UNION
SELECT City FROM Suppliers
ORDER BY City;
```

```
SELECT City FROM Customers
UNION ALL
SELECT City FROM Suppliers
ORDER BY City;
```

INTERSECT

2 sorgu sonucunda dönen veri setini karşılaştırıp sadece ORTAK olan verilerini gösterir

Önemli noktalar:

- Alt alta gelen kolonlar aynı veri tipinde olmalı
- * (star) ile yapılan union işlemlerinde kolon kaymaları olabilir. O nedenle kolon isimlerini yazmanız önerilir.

```
SELECT City FROM Customers
INTERSECT
SELECT City FROM Suppliers
ORDER BY City;
```


EXCEPT – EXCEP ALL

2 sorgu sonucunda dönen veri setini karşılaştırarak 1. veri setinde olup 2.'de olmayan kayıtları listeler.

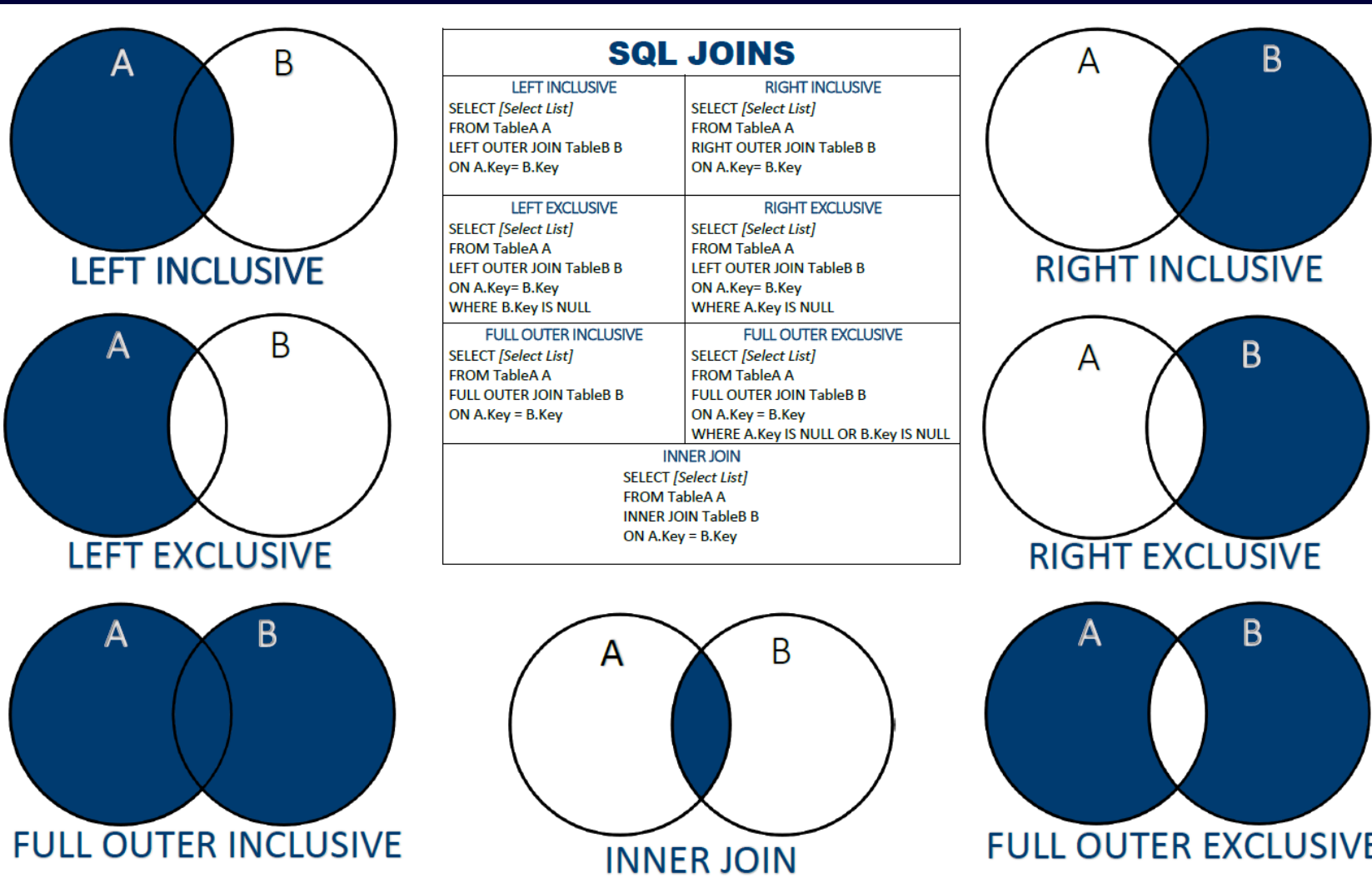
Önemli noktalar:

- Alt alta gelen kolonlar aynı veri tipinde olmalı
- * (star) ile yapılan union işlemlerinde kolon kaymaları olabilir. O nedenle kolon isimlerini yazmanız önerilir.
- EXCEPT işlemi tekil değer döndürür EXCEPT ALL ise veri içeriğine göre tekrarlı veri döndürebilir

```
SELECT City FROM Customers
EXCEPT
SELECT City FROM Suppliers
ORDER BY City;
```

```
SELECT City FROM Customers
EXCEPT ALL
SELECT City FROM Suppliers
ORDER BY City;
```

JOINS



INNER JOIN

2 veri setinin belirli bir kolon üzerinden birleştirmek için kullanılır. Sadece eşleşen veriler gösterilir.

```
SELECT testproduct_id, product_name, category_name
FROM testproducts
INNER JOIN categories
ON testproducts.category_id = categories.category_id;
```

testproduct_id	product_name	category_name
1	Johns Fruit Cake	Confections
6	Janes Favorite Cheese	Dairy Products
8	Ellas Special Salmon	Seafood
9	Roberts Rich Spaghetti	Grains/Cereals

(4 rows)

LEFT JOIN

2 veri setinin belirli bir kolon üzerinden birleştirmek için kullanılır.

1. yazılan tablonun tüm verileri listelenir, 2. yazılan tablodaki veriler ise sadece eşleşenler gösterilir.

```
SELECT testproduct_id, product_name, category_name
FROM testproducts
LEFT JOIN categories
ON testproducts.category_id = categories.category_id;
```

testproduct_id	product_name	category_name
1	Johns Fruit Cake	Confections
2	Marys Healthy Mix	
3	Peters Scary Stuff	
4	Jims Secret Recipe	
5	Elisabeths Best Apples	
6	Janes Favorite Cheese	Dairy Products
7	Billys Home Made Pizza	
8	Ellas Special Salmon	Seafood
9	Roberts Rich Spaghetti	Grains/Cereals
10	Mias Popular Ice	

(10 rows)

RIGHT JOIN

2 veri setinin belirli bir kolon üzerinden birleştirmek için kullanılır. 2. yazılan tablonun tüm verileri listelenir, 1. yazılan tablodaki veriler ise sadece eşleşenler gösterilir.

```
SELECT testproduct_id, product_name, category_name
FROM testproducts
RIGHT JOIN categories
ON testproducts.category_id = categories.category_id;
```

testproduct_id	product_name	category_name
1	Johns Fruit Cake	Confections
6	Janes Favorite Cheese	Dairy Products
8	Ellas Special Salmon	Seafood
9	Roberts Rich Spaghetti	Grains/Cereals
		Condiments
		Meat/Poultry
		Beverages
		Produce

(8 rows)

FULL JOIN

2 veri setinin belirli bir kolon üzerinden birleştirmek için kullanılır.

1. ve 2. yazılan tablonun tüm verileri listelenir,

```
SELECT testproduct_id, product_name, category_name
FROM testproducts
FULL JOIN categories ON testproducts.category_id =
categories.category_id;
```

testproduct_id	product_name	category_name
1	Johns Fruit Cake	Confections
2	Marys Healthy Mix	
3	Peters Scary Stuff	
4	Jims Secret Recipe	
5	Elisabeths Best Apples	
6	Janes Favorite Cheese	Dairy Products
7	Billys Home Made Pizza	
8	Ellas Special Salmon	Seafood
9	Roberts Rich Spaghetti	Grains/Cereals
10	Mias Popular Ice	
		Condiments
		Meat/Poultry
		Beverages
		Produce

(14 rows)

CROSS JOIN

1. veri setinin tüm kayıtlarını 2. veriseti ile teker teker eşleştirerek sonucu döndürür.

```
SELECT testproduct_id, product_name, category_name
FROM testproducts
CROSS JOIN categories;
```

testproduct_id	product_name	category_name
1	Johns Fruit Cake	Beverages
1	Johns Fruit Cake	Condiments
1	Johns Fruit Cake	Confections
1	Johns Fruit Cake	Dairy Products
1	Johns Fruit Cake	Grains/Cereals
1	Johns Fruit Cake	Meat/Poultry
1	Johns Fruit Cake	Produce
1	Johns Fruit Cake	Seafood
2	Marys Healthy Mix	Beverages
2	Marys Healthy Mix	Condiments
2	Marys Healthy Mix	Confections
2	Marys Healthy Mix	Dairy Products
2	Marys Healthy Mix	Grains/Cereals
2	Marys Healthy Mix	Meat/Poultry
2	Marys Healthy Mix	Produce
2	Marys Healthy Mix	Seafood
3	Peters Scary Stuff	Beverages
3	Peters Scary Stuff	Condiments

DDL – CREATE VIEW

Çok kullanılan sorguları DB üzerinde kaydedip kullanmak için kullanılır.

```
CREATE OR REPLACE VIEW comedies AS
SELECT *
FROM films
WHERE kind = 'Comedy';

CREATE OR REPLACE VIEW pg_comedies AS
SELECT *
FROM comedies
WHERE classification = 'PG'
WITH CASCADE CHECK OPTION;

CREATE RECURSIVE VIEW nums_1_100 (n) AS
VALUES (1)
UNION ALL
SELECT n+1 FROM nums_1_100 WHERE n < 100;
```




TEŞEKKÜRLER